# R_sine — How to Present to Your Guide

A slide-by-slide speaking script, likely questions with answers, and presentation tips. Read this once before you present; keep it beside you during.

## Before you start — 7 tips

- Total time: aim for 8–10 minutes of talking, then questions. About 40 seconds per slide.
- Open with the goal and the demo picture so the guide sees the result first, then explain how.
- Lead with your OWN work (the DH derivation, the Jacobian, the IK); mention ikpy only as a check.
- When a slide shows numbers, say the one-line takeaway ('matches to machine precision'), not every digit.
- Point at the figure while you talk — the drawn sine and the workspace are your strongest visuals.
- If you don't know an answer, say what you'd do to find out — that reads as maturity, not weakness.
- Keep the live RViz demo command ready as a backup: ros2 launch Rsine rsine_sine.launch.py

## Slide-by-slide script

### Slide 1 — R_sine — Drawing a Sine Wave with a myCobot 280

**Say:** Start here. 'My project makes a 6-DOF myCobot 280 arm draw a sine wave on a board. What makes it rigorous is that I derived the kinematics myself using the modified DH method from our course, built the Jacobian by velocity propagation, and solved the inverse kinematics with damped least squares — then I verified every piece against an independent library. It all runs in ROS 2.'

**If asked:** Why a sine wave?

**Answer:** It is a smooth, continuously-curving path that exercises all axes and makes tracking accuracy easy to see — a good test of the full FK/Jacobian/IK pipeline.

### Slide 2 — Objective and Method

**Say:** 'The project is built around the three standard problems: forward kinematics, the Jacobian, and inverse kinematics. I generate the sine as a list of 3D target points, solve inverse kinematics for each point to get joint angles, and play that back on the robot.'

**If asked:** What is the difference between forward and inverse kinematics?

**Answer:** Forward: joints -> pose (one unique answer). Inverse: pose -> joints (can have several answers or none if unreachable).

### Slide 3 — Modified (Craig) DH Convention

**Say:** 'I use the modified DH convention from class. Each joint is described by four numbers — twist, length, offset, and angle — and each link transform is this product of a rotation and translation about x then z. Multiplying them gives the forward kinematics.'

**If asked:** Why modified DH and not standard DH?

**Answer:** Modified DH places the frame at the proximal joint, which many textbooks (Craig) use; it is the convention in my reference notes. Both are valid.

### Slide 4 — Identified DH Table (verified to 1e-15 m)

**Say:** 'This table is the heart of the model. The official URDF doesn't come with DH parameters, so I identified them by fitting the four numbers per joint until my DH forward kinematics matched the URDF exactly — the error is 1e-15 meters, essentially machine zero. So the table is provably correct, not guessed.'

**If asked:** How do you know the table is right?

**Answer:** Its forward kinematics matches the manufacturer URDF to ~1e-15 m over 5000 random joint configurations, and also matches ikpy to ~1e-16 m.

### Slide 5 — Forward Kinematics

**Say:** 'Forward kinematics is just the product of the six link transforms. Here's the arm at its home pose with the frame at every joint. When I feed the same joints to ikpy, it lands on the identical position — so my FK is correct.'

**If asked:** What frame is the tool?

**Answer:** link6_flange — the output flange where a pen or gripper mounts.

## Slide 6 Jacobian — Velocity Propagation

**Say:** 'For the Jacobian I used the velocity-propagation method from the notes — you push the angular and linear velocity outward joint by joint. For revolute joints it simplifies to this cross-product form. I checked it against a numerical Jacobian and it agrees to seven decimal places.'

**If asked:** What is the Jacobian used for here?

**Answer:** It drives the inverse kinematics — each IK step multiplies a damped inverse of J by the pose error to update the joint angles.

## Slide 7 Reachable Workspace (verified)

**Say:** 'This is the reachable workspace — every point the tool can reach — from three views. The red sine is my drawing target, and you can see it sits comfortably inside. This is also how I chose where to put the board: somewhere the arm can comfortably reach.'

**If asked:** Why not draw further out or on the table?

**Answer:** This is a small arm; keeping the pen perpendicular to a far or downward surface exceeds its reach. The vertical board close in is fully reachable.

## Slide 8 Inverse Kinematics — Damped Least Squares

**Say:** 'I solve inverse kinematics with damped least squares. Each iteration nudges the joints by a damped inverse of the Jacobian times the pose error. The damping is what keeps it stable near singularities. It converges in about 16 steps per point and always respects the joint limits.'

**If asked:** Why damped, not the plain Jacobian inverse?

**Answer:** Near singularities the plain inverse blows up. Damping trades a negligible position error for numerical stability.

## Slide 9 Configurable Drawing Plane

**Say:** 'The plane is general. I define it by an origin and two axes — one to advance along, one for the wave. The pen is kept perpendicular to the surface. So the same code draws on a vertical board or a flat plate; I just change the parameters at launch.'

**If asked:** Could it draw a different shape?

**Answer:** Yes — only the path generator changes; the FK/Jacobian/IK pipeline is the same.

## Slide 10 Result — Drawing the Sine

**Say:** 'And here's the result — the arm drawing the two-cycle sine, shown as a progression. The red line is what the tool actually traces. It matches the intended curve to under a millimetre, and the pen stays perpendicular to the board the entire time.'

**If asked:** Is this simulation or the real robot?

**Answer:** Simulation in RViz with the exact robot model; the design is hardware-ready since it uses the real URDF joint names and limits.

## Slide 11 Accuracy and Verification

**Say:** 'These are the numbers I'd stake the project on. Forward kinematics is exact to machine precision, the Jacobian to seven digits, and the drawn path tracks to about a micron. The error plot stays a thousand times under a millimetre.'

**If asked:** 0.001 mm seems too good — is it real?

**Answer:** It is the numerical tracking error of the solver in simulation. On real hardware, encoder resolution and calibration would dominate; this bounds the algorithm's own contribution.

## Slide 12 Analytical vs ikpy

**Say:** 'Finally, a sanity check against an independent library, ikpy. Every comparison — forward kinematics, Jacobian, inverse kinematics — agrees far below a millimetre, and both methods draw the same sine. Two independent implementations agreeing is my strongest evidence the math is right.'

**If asked:** If ikpy exists, why write your own?

**Answer:** To understand and control the math, handle orientation and limits my way, and have a solver that matches the course methodology — with ikpy only as a check.

## Slide 13 ROS 2 Architecture

**Say:** 'Software-wise, the math is in plain Python modules with no ROS, so I can unit-test them. Then two small ROS nodes: one streams the joint trajectory, the other records the drawn line. A single launch file brings up the robot and RViz.

**If asked:** Why precompute the trajectory instead of solving live?

**Answer:** The sine is fixed, so solving once and replaying is smooth and cheap; live IK would just repeat the same work every loop.

## Slide 14 Conclusion and Future Work

**Say:** 'To conclude: I built the kinematics from scratch, verified every stage, and used it to draw a sine on a board in ROS 2. It generalizes to other surfaces and shapes. Next I'd move to the physical arm with a real pen and add Gazebo physics. Thank you — happy to take questions.'

**If asked:** What was the hardest part?

**Answer:** Identifying a correct, physically clean DH table from a URDF that isn't in DH form — solved by fitting and verifying to machine precision.